

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
MAROSVÁSÁRHELYI KAR,
SZOFTVERFEJLESZTÉS SZAK**



SAPIENTIA
ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM

Echo: AI-alapú nyelvtanuló alkalmazás fejlesztése és oktatási
célú értékelése

DIPLOMADOLGOZAT

Témavezető:
Dr. Buza Krisztián,
Egyetemi docens

Végzős hallgató:
Suciu Aksza

2026

**UNIVERSITATEA SAPIENTIA DIN CLUJ-NAPOCA
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
SPECIALIZAREA DEZVOLTAREA APLICAȚIILOR
SOFTWARE**



**UNIVERSITATEA
SAPIENTIA**

Echo: Development and Educational Evaluation of an AI-Based
Language Learning Application

LUCRARE DE DIPLOMĂ

Coordonator științific:
Dr. Buza Krisztián,
Lector universitar

Absolvent:
Suciu Aksza

2026

**SAPIENTIA HUNGARIAN UNIVERSITY OF
TRANSYLVANIA
FACULTY OF TECHNICAL AND HUMAN SCIENCES
SOFTWARE ENGINEERING SPECIALIZATION**



SAPIENTIA
HUNGARIAN UNIVERSITY
OF TRANSYLVANIA

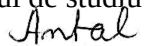
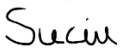
Echo: Dezvoltarea și evaluarea educațională a unei aplicații de
învățare a limbilor stăine bazate pe inteligență artificială

BACHELOR THESIS

Scientific advisor:
Dr. Buza Krisztián,
Lecturer

Student:
Suciu Aksza

2026

UNIVERSITATEA „SAPIENTIA” din CLUJ-NAPOCA Facultatea de Științe Tehnice și Umaniste din Târgu Mureș Specializarea: Informatică		Viza facultății:
LUCRARE DE DIPLOMĂ		
Coordonator științific: Lector dr. Márton Gyöngyvér	Candidat: Suciu Aksza Anul absolvirii: 2024	
a) Tema lucrării de licență: DineEase: SISTEM INOVATOR DE REZERVARE PENTRU RESTAURANTE		
b) Problemele principale tratate: • Studiul bibliografic al aplicațiilor similare • Tratarea problemelor de siguranță: autentificare și autorizare • Prezentarea tehnologiilor utilizate • Prezentarea specificațiilor sistemului creat: cerințe utilizator, cerințe sistem • Arhitectura sistemului creat, baza de date, interfețele aplicației, urmărirea versiunilor, proiect management		
c) Desene obligatorii: • Schema bloc a aplicației • Schema bazei de date • Diagramele use-case		
d) Softuri obligatorii: .NET: pentru backend MSSQL: pentru baza de date Flutter: pentru frontend		
e) Bibliografia recomandată: Mark Stamp. <i>Information security: principles and practice</i> . John Wiley & Sons, 2011 Microsoft Corporation. Asp.net core documentation. https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-7.0 Microsoft Corporation. Microsoft sql server documentation. https://learn.microsoft.com/en-us/sql/?view=sql-server-ver16 Google LLC. Flutter documentation. https://docs.flutter.dev/		
f) Termene obligatorii de consultații: săptămânal		
g) Locul și durata practicii: Universitatea „Sapientia” din Cluj-Napoca, Facultatea de Științe Tehnice și Umaniste din Târgu Mureș Primit tema la data de: 1 mai 2023 Termen de predare: 24 iunie 2024		
Semnătura Director Departament 	Semnătura coordonatorului 	
Semnătura responsabilului programului de studiu 	Semnătura candidatului 	

Declarație

Subsemnatul/a SUCIU AKSZA, absolvent(ă) al/a
specializării INFORMATICA, promoția 2024.
cunoscând prevederile Legii învățământului superior nr. 199 din 2023 și a Codului de etică și
deontologie profesională a Universității Sapiientia cu privire la furt intelectual declar pe propria
răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea
personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din
literatura de specialitate sunt citate în mod corespunzător.

Localitatea, CORUNCA

Data: 13.06.2024.

Absolvent

Semnătura.....Suciu.....

Kivonat

A dolgozatom egy mesterséges intelligenciával támogatott beszédfejlesztő alkalmazás tervezését és megvalósítását mutatja be. A hagyományos nyelvtanulási módszerek gyakran a nyelvtani szabályok és az elszigetelt szókincs elsajátítására helyezik a hangsúlyt, miközben korlátozott lehetőséget biztosítanak a valós kommunikáció gyakorlására. Ennek következtében a tanulók nehezebben alkalmazzák tudásukat valódi beszédhelyzetben.

A fejlesztett alkalmazás célja, hogy beszélgetésalapú tanulási környezetet biztosítson, ahol a felhasználók mesterséges intelligenciával folytatott párbeszéd során fejleszthetik nyelvi készségeiket. Az AI automatikusan felismeri, kijavítja a hibáikat, majd segít nekik a fejlődésben azzal hogy ismételten előkerülnek ezek a beszélgetések során.

A dolgozat bemutatja az alkalmazás felépítését, a megvalósítás során alkalmazott módszereket és technológiai megoldásokat, valamint azt, hogy a beszélgetésközpontú, hibaalapú tanulási megközelítés miként járult hozzá a beszédképesség hatékony fejlesztéséhez.

Rezumat

Lucrarea mea prezintă proiectarea și implementarea unei aplicații de dezvoltare a abilităților de vorbire, susținută de inteligență artificială. Metodele tradiționale de învățare a limbilor străine pun adesea accentul pe însușirea regulilor gramaticale și a vocabularului izolat, oferind în același timp oportunități limitate pentru exersarea comunicării reale. Ca urmare, cursanții întâmpină dificultăți în aplicarea cunoștințelor dobândite în situații reale de vorbire.

Scopul aplicației dezvoltate este de a oferi un mediu de învățare bazat pe conversație, în care utilizatorii își pot dezvolta competențele lingvistice prin dialoguri purtate cu o inteligență artificială. AI-ul identifică și corectează automat greșelile utilizatorilor și sprijină procesul de învățare prin reluarea acestora în cadrul conversațiilor ulterioare.

Lucrarea prezintă structura aplicației, metodele și soluțiile tehnologice utilizate în procesul de implementare, precum și modul în care abordarea de învățare centrată pe conversație și pe greșeli contribuie la dezvoltarea eficientă a abilităților de comunicare orală.

Abstract

This thesis presents the design and implementation of an artificial intelligence-supported application for the development of speaking skills. Traditional language learning methods often emphasize the acquisition of grammatical rules and isolated vocabulary, while providing limited opportunities for practicing real-life communication. As a result, learners face difficulties in applying their knowledge in authentic speaking situations.

The aim of the developed application is to provide a conversation-based learning environment in which users can improve their language skills through dialogues with an artificial intelligence. The AI automatically identifies and corrects users' errors and supports their progress by revisiting these mistakes during subsequent conversations.

The thesis describes the structure of the application, the methods and technological solutions applied during its implementation, and examines how a conversation-centered, error-based learning approach contributes to the effective development of speaking skills.

Tartalomjegyzék

1. Bevezető	11
2. Hasonló alkalmazások, felhasznált technológiák	12
2.1. Hasonló alkalmazások	12
2.2. Felhasznált technológiák	13
2.2.1. Flutter	13
2.2.2. ASP.NET Core	13
2.2.3. Microsoft SQL Server	14
2.2.4. Python	14
3. Rendszer specifikációja	15
3.1. Felhasználói követelmények	15
3.2. Rendszerkövetelmények	17
3.2.1. Funkcionális követelmények	17
3.2.2. Nem funkcionális követelmények	19
4. Tervezés	21
4.1. Szoftver terv	21
4.1.1. Logikai nézet	21
4.1.2. Folyamat nézet	22
4.2. Adatbázis terv	24
4.3. Felhasználói felület terv	25
4.4. Verziókövetés	26
4.5. Projektmenedzsment	27
5. Megvalósítás	29
5.1. Az alkalmazás architektúrája	29
5.2. Frontend megvalósítása	29
5.3. Backend megvalósítása	31
5.4. AI service megvalósítása	32
6. Kutatási terv és értékelési módszertan	34
6.1. A kutatás célja	34
6.2. Kutatási kérdések és hipotézisek	35
6.3. Résztvevők	35
6.4. A tanóra témája és menete	36
6.5. Mérőeszközök	36

6.6. Adatelemzési módszerek	36
6.7. Etikai szempontok	37
7. Kutatási eredmények és kiértékelés	38
8. Következtetések és továbbfejlesztési lehetőségek	39
Ábrák jegyzéke	40
Irodalomjegyzék	44

1. fejezet

Bevezető

Sok nyelvtanuló szembesül azzal a problémával, hogy bár hosszú időn keresztül tanul egy idegen nyelvet, mégsem érzi magát magabiztosnak, amikor valódi beszédhelyzetbe kerül. Gyakran előfordul, hogy az elsajátított tudása ellenére nehézséget jelent egy egyszerű párbeszéd lebonyolítása is, vagy a gondolatok megfogalmazása. Sokszor hallhatjuk azt is, hogy egy nyelvet akkor lehet igazán megtanulni, ha azt aktívan használjuk [Swa85, Lon96], azonban erre nem minden tanulónak van lehetősége. Különösen igaz ez azokra, akiknek a környezetükben nincs olyan személy, akivel rendszeresen gyakorolhatnák a nyelvet.

A hagyományos nyelvtanulási módszerek és alkalmazások jellemzően nyelvtani szabályokra, elszigetelt szókincs elsajátítására helyezik a hangsúlyt. Bár ezek fontos elemei a nyelvtanulásnak, gyakran kevés figyelem jut a beszédképesség fejlesztésére és a valós kommunikáció gyakorlására. [Swa85, Lon96]

Ezekre a problémákra szeretnék megoldást nyújtani az általam fejlesztett alkalmazással, amely egy mesterséges intelligenciával támogatott beszédalapú nyelvtanulási környezetet biztosít. Az alkalmazás lehetőséget nyújt arra, hogy a felhasználók mesterséges intelligenciával folytatott párbeszéd során gyakorolhassák a célnyelvet, mintha egy valódi beszélgetőpartnerrel kommunikálnának. [XZS⁺24] Az AI nemcsak részt vesz a beszélgetésben, hanem felismeri és kijavítja a nyelvi hibákat [LSS13, SL12], illetve eltárolja azokat, későbbi feldolgozás céljából.

A rendszerben külön figyelem irányul arra, hogy a felhasználók ne csupán javítást kapjanak, hanem lehetőséget arra is, hogy saját hibáikból tanuljanak. A gyakran előforduló hibák ismételten megjelennek a tanulási folyamat, beszélgetések és különböző gyakorlási módszerek segítségével, így elősegítve a helyes nyelvi minták rögzülését és a beszédképesség folyamatos fejlődését.

A következő fejezetekben bemutatom az alkalmazás tervezésének és megvalósításának folyamatát, az alkalmazott módszerek és technológiai megoldásokat, valamint elemzem, hogy a beszélgetésközpontú, hibaalapú megközelítés miként járul hozzá a hatékony nyelvtanuláshoz és a beszédképesség fejlesztéséhez.

2. fejezet

Hasonló alkalmazások, felhasznált technológiák

2.1. Hasonló alkalmazások

Dolgozatom ezen részében először megvizsgálunk néhány olyan alkalmazást, amelyek hasonló feladatot végeznek, mint az általam megvalósított alkalmazásunk.

A piacon található néhány nyelvtanuló alkalmazás, amelyek hasonló funkcionalitásokkal vannak ellátva, mint az Echo alkalmazás. Dolgozatom során megvizsgáltam több alkalmazást is, és elmondható róluk hogy felépítésük hasonló, ugyanakkor az Echo alkalmazás egyedisége e beszéd fókuszú tanulásban rejlik.

A vizsgált alkalmazások között szerepelnek a következők:

[Speak: Language Learning \[Spe26a\]](#), [Praktika: AI learning tutor \[Pra26b\]](#), [Duolingo\[Duo23\]](#).

Ezek az alkalmazások mind nyelvtanítással foglalkoznak, különböző megközelítésekben. A Speak és a Praktika leginkább beszédfejlesztésre fókuszál, különböző szituációs beszélgetéseket ad a felhasználónak, amellyel támogatja a kommunikációs készségét az idegen nyelven. A Duolingonak is van egy hasonló funkciója, amelyben kommunikációt lehet fejleszteni, viszont ez az alkalmazás inkább szókincs bővítésre, nyelvtani szabályok elsajátítására használatos.

A Speak alkalmazás AIal való beszélgetést támogat, különböző témakörökben. Ez képes elemezni a beszédet, visszajelzést ad a kiejtésről, javaslatokat ad mondat szerkesztésre. [\[Spe26b\]](#) Ez az alkalmazás a leghasonlóbb az általunk fejlesztett alkalmazáshoz.

A Praktika ugyanúgy a mesterséges intelligencia segítségét hívja segítségül a nyelvtanulásban, viszont ebben az alkalmazásban különböző avatarokkal beszélgethetük, így még személyesebbé és még realisabbá téve a kommunikációt a felhasználó számára. [\[Pra26a\]](#)

A Duolingo a legelterjedtebb alkalmazás, amely elsősorban szókincs bővítésére hasznos. [\[Duo23\]](#) Játékos felülete vonzóvá teszi a felhasználónak az alkalmazás használatát. A beszélgető mód a fizetős verzióban érvényes, itt is mesterséges intelligencia használatával, viszont a hangsúly az alkalmazásnak nem feltétlenül ezen van.

A vizsgált alkalmazások közös jellemzője természetesen az, hogy mindegyik a nyelvtanulást támogatja és van nekik AIal támogatott verziójuk is. Legtöbbüknél ez a funkció fizetős. Ezekben az alkalmazásokban ugyanúgy lehet profilt készíteni, támogatják azt,

hogy nap mint nap gyakorolja a felhasználó az adott nyelvet, illetve statisztikákat is megtekinthetnek a fejlődésükről.

Ezekben az alkalmazásokban fellelhető a jól ismert gamifikáció is. Ennek célja az, hogy a felhasználó játszva tanuljon, jól érezze magát, jutalmazva érezze magát tanulás közben. [DSN⁺11] Ezzel a módszerrel könnyebben meggyőzhetik a fejlesztők a felhasználókat, hogy szívesebben használják az alkalmazást, ezzel motiválva őket a fejlődésre.

Összességében elmondható hogy ezek az alkalmazások különböző igényekhez mérten, hatékony megoldásokat kínálnak a nyelvtanulásra. A legtöbb alkalmazás vagy gamifikációra vagy előre meghatározott feladatokkal segíti a tanulást. A megvalósított alkalmazás célja ezzel szemben az, hogy a felhasználók számára egy olyan platformot biztosítson, ahol a hangsúly elsősorban a valós idejű beszélgetésen és kommunikációs készségek fejlesztésén van mesterséges intelligencia segítségével.

2.2. Felhasznált technológiák

Az előzőleg felsorolt alkalmazások nagyrészt hasonló technológiákat használnak. Mobil frontendnek a React Native nyelvet használják, cloud infrastructure, illetve NLP modelleket használnak valós idejű hangfeldolgozás és generálás céljából. Egyetlen eltérés a Duolingo alkalmazásnál van. Lévén, hogy egy régebbi alkalmazásról beszélünk, inkább az akkori trendeket követve másfajta technológiákat használt, ellenben az előzőleg felsorolt alkalmazásokkal, amelyek nem olyan rég készültek. A Duolingo Kotlin-t használ modern Android fejlesztés céljából, a backend technológiája pedig a népszerű Python nyelven íródott, amely könnyen használható machine learninghez. AI szempontjából vizsgálva GPT 4 integrációt használtak fel az alkalmazásban, amivel biztosítják az interaktívabb feladatokat és valós idejű beszélgetésekhez használják.

A következő alfejezetekben röviden bemutatom az általam felhasznált technológiákat.

2.2.1. Flutter

A Flutter Google által fejlesztett nyílt forráskódú keretrendszer, amelyet UI fejlesztésére használnak. Ez lehetővé teszi a keresztplatformos alkalmazások fejlesztését egyetlen kódbázisból. A projekt frontendje ebben a keretrendszerben valósult meg, mivel modern felhasználói felületet biztosít különböző csomagok segítségével, illetve jól integrálható backend szolgáltatással. [Goo26]

2.2.2. ASP.NET Core

Az ASP.NET Core nyílt forráskódú webes keretrendszer, amelyet a Microsoft hozott létre. A keretrendszer segítségével komplex alkalmazások backendjét hozhatjuk létre, amelyek gyorsak és skálázhatók. Ez olyan API tervezéséhez és elkészítéséhez járul hozzá, amely segít hatékonyan adatáramlást elérni az adatbázis vagy az AI modell és a kliensalkalmazás között. [Mic26c]

2.2.3. Microsoft SQL Server

A Microsoft SQL Server (MSSQL) relációs adatbáziskezelő rendszer, amely nagy mennyiségű adatok tárolását és kezelését teszi lehetővé. Ez az adatbáziskezelő nagyon egyszerűen integrálható a .NET keretrendszerrel, lévén hogy Microsoft által fejlesztették őket. [Mic26d]

2.2.4. Python

A Python magasszintű programozási nyelv, amely széles körben elterjedt, leginkább mesterséges intelligencia és gépi tanulás területén. Az alkalmazás AI funkciói külön Python alapú mikroszolgáltatásokként kerültek megvalósításra. [Pyt26]

A felhasznált AI-ok a következőkben lesznek bemutatva

Faster-Whisper

A Whisper beszédfelismerő modell optimalizált változata ez. Ez hozzájárul az alkalmazásban a felhasználó hangüzeneteinek szöveggé alakítására, amikor konverszációt kezdeményez a felhasználó, az ő beszédét alakítja át írásos formába. [SYS26, RKX⁺22]

Phi3-Mini

A Phi3 egy kisméretű nyelvi modell, amely természetes nyelvű válaszok generálására és szövegfeldolgozásra alkalmas. A rendszerben ez végzi a beszélgetések kezelését, valamint nyelvi hibák felismerését és javítását. [AJA⁺24]

Piper TTS

A Piper egy nyílt forráskódú text to speech rendszer, amely képes a szöveget beszéddé alakítani. Az alkalmazásban az AI által generált válaszok hangos felolvasására szolgál. [Rha26]

3. fejezet

Rendszer specifikációja

3.1. Felhasználói követelmények

Egyetlen felhasználói típusunk van az applikációban: a tanuló, amely nyelvi készségeit fejleszteni kívánja.

- **Platform**

Egy mobilalkamazás formájában a tanulni vágyó felhasználók mesterséges intelligencia segítségével gyakorolhatják a célnyelvet, ahol fejlődésüket tudják követni, különböző gyakorló feladatokat végezni és a beszédkésztségükön dolgozni.

- **Regisztráció oldal**

A felhasználó a szükséges adatok megadásával fiókot hozhat létre a platformon. Ennek elvégzése után kaphat hozzáférést az alkalmazás lényegi funkcióihoz.

- **Bejelentkezés oldal**

A már létrehozott fiókba bejelentkezhet a felhasználó az email cím és jelszó megadásával. Sikeres bejelentkezés után pedig máris nekiláthat a nyelvtanulásnak.

- **Onboarding oldal**

Az első bejelentkezés után a felhasználó kiválaszthatja a tanulni kívánt nyelvet, tanulási célját és egy kis felmérés után személyre szabott rendszert használhatja.

- **Főoldal**

A főoldalon a felhasználó több gombbal is találkozhat, amelyek különböző oldalakra vezetnek. Itt megtekintheti a legutóbbi beszélgetéseket, napi ajánlásokat, hibaösszegzéseket, illetve egy gomb segítségével azonnal beszélgetést indíthat a mesterséges intelligenciával. Ugyanitt láthatja a profil oldalát is illetve a gyakorlást indító gombot is. Az oldal célja, hogy egy egységes helyen megtalálhassa a fő funkciókat a felhasználó.

- **Napi ajánlás oldal**

Személyre szabott ajánlásokkal találkozhatnak ezen az oldalon a felhasználók. Ez tartalmaz gyakorló feladatokat, ismétlődő hibákat, ajánlott beszélgetési témákat.

- **Előzmények**

Ezen az oldalon visszakövetheti a felhasználó korábbi beszélgetési témáit, hibáit ezekben a beszélgetésekben, összefoglalókat róla. Ez segít az ismétlésben, és a fejlődés megfigyelésében is.

- **Hibák összegzése, Szótár**

Ez az oldal arra szolgál, hogy a felhasználó megtekintse korábbi hibáit, javításokat, magyarázatokat. Segíti a tudatos tanulást az, hogy repetitíven tanulhat a felhasználó.

- **Beszélgetés oldal**

A platform a beszédfejlesztésre fókuszál leginkább, így elsősorban beszédben kommunikálhat a mesterséges intelligenciával különböző témákban. Ez feldolgozza a beszédet, válaszol, javítja a hibákat. Opcionálisan találunk itt egy chat felületet, ahol esetlegesen az AI által meg nem értett szavakat vagy mondatokat gépelhet a felhasználó és erre kap választ. Azt fontos kiemelni, hogy ezen az oldalon elérhető egy gomb, amellyel feliratozhatja a mesterséges intelligencia válaszait, vagy akár a sajátjait is.

- **Gyakorlás oldal**

Több fajta gyakorlásból választhat a felhasználó, amely segíti úgy a beszédképességét mint az írás- olvasás készségeit is. Flashcard vagy Pronunciation card funkcióval a felhasználó megtanulhatja különböző szavak kiejtését és jelentését is. Correct the sentence vagy Fill in the gap funkcióval gyakorolhatja a helyes beszédet, ezen az oldalon felhasználva vannak a tanuló hibái is, ahhoz hasonló mondatokat kell kijavítania.

- **Profil**

A felhasználó ezen az oldalon megtekintheti adatait, fejlődését, változtathat különböző személyes információin.

- **Statisztikák**

Grafikus formában is elérhetőek a tanulók fejlődési adatai, hibái, aktivitása.

- **Beállítások, Mesterséges intelligencia személyre szabása**

Ezen az oldalon lehetősége nyílik a felhasználónak módosítani tanulását. Változtathat itt nyelvi, illetve AI beállításokat.

3.2. Rendszerkövetelmények

3.2.1. Funkcionális követelmények

Az alábbiakban bemutatok minden fontos funkciót az alkalmazásból.

- **Regisztráció**

A rendszer lehetőséget biztosít a felhasználó számára egy új fiók létrehozására. A regisztráció során a felhasználónak meg kell adnia a szükséges adatait. A rendszer ellenőrizni az adatokat, és hogy ki van-e töltve minden mező, formátumot ellenőriz. Ezek után ha hibát észlel a rendszer, akkor hibaüzenetet jelenít meg. A sikeres regisztráció esetén a felhasználó adatai biztonságosan eltárolásra kerülnek az adatbázisban, titkosított formában.[OWA26]

- **Bejelentkezés**

Ha már létrehozott egy fiókot a felhasználó, akkor lehetősége nyílik bejelentkezni a meglévő fiókjába. A felhasználónak meg kell adnia email címét és jelszavát. Nem megfelelő adatok esetén visszajelzést küld a hibáról a rendszer, sikeres bejelentkezés esetén pedig a rendszer hitelesítési tokent generál [JBS15], amely lehetővé teszi a felhasználó számára az alkalmazás további, főbb funkcióit.

- **Beszélgetés a mesterséges intelligenciával**

Az applikáción belül a felhasználóknak van lehetőségük a mesterséges intelligenciával folytatott beszélgetésre, a célnyelven. A felhasználó hang alapján beszélgethet, vagy akár néhány mondatot írásban is küldhet a mesterséges intelligenciának. A rendszer célja, hogy a beszélgetés minél inkább hasonlítson egy valós kommunikációs helyzethez. Ennek segítségével a felhasználó tanulhatja a magabiztosabb kommunikációt.

- **Hibák felismerése és kezelése**

A beszélgetés során a rendszer automatikusan felismeri a felhasználó által elkövetett nyelvi hibákat. Ezeket a hibákat kijavítja, majd eltárolja az adatbázisban, kiegészítve a helyes megoldással és szükség esetén magyarázattal. A hibák később visszakereshetők és felhasználhatók a tanulási folyamat támogatására.

- **Gyakorló feladatok generálása**

A rendszer képes a felhasználó korábbi hibái alapján személyre szabott gyakorló feladatok generálására. Ezek a feladatok különböző típusúak lehetnek, például hiányos mondatok, kiejtés tanulás, szókincs alapú feladatok. A cél, hogy a felhasználó ismételten találkozzon a hibáival, és ezáltal hatékonyabban rögzítse a helyes nyelvi formákat.

- **Előzmények kezelése**

A rendszer nem tárolja el a felhasználók korábbi beszélgetéseit, biztonsági okokból, viszont eltárolja a beszélgetések témájának címét, összefoglalót róla, és a hibákat, megjegyzéseket. Ennek segítségével a felhasználó nyomon tudja követni fejlődését és visszatérhet korábbi tanulási helyzetekhez.

- **Statisztikák**

A rendszer különböző statisztikai adatokat gyűjt és jelenít meg a felhasználó számára, mint például a beszélgetések száma, hibák mennyisége, fejlődés üteme. Ezek az adatok segítik a felhasználót abban, hogy jobban átlássa a tanulási folyamatát.

- **Profil**

A felhasználó megtekintheti és módosíthatja saját adatait a profil oldalon. Itt lehetősége nyílik a személyes beállítások, például a nyelvi szint vagy a tanulási célok módosítására.

- **Mesterséges intelligencia beállítások kezelése**

A rendszer lehetőséget biztosít különböző beállítások módosítására, mint például az alkalmazás nyelve, az értesítések kezelése, az AI viselkedésének testreszabása. Ezek a beállítások hozzájárulnak a felhasználói élmény személyre szabásához.

3.2.2. Nem funkcionális követelmények

Az applikáció mobil alkalmazásnak lett elkészítve, ugyanakkor a Flutter [Goo26] keretrendszernek köszönhetően márs platformokon is használható. Gyors és stabil nyelvtanulási környezet biztosítása a cél, amely támogatja a mesterséges intelligenciával történő kommunikációt és beszédfejlesztést.

- **Operációs rendszer**

Az alkalmazás Android operációs rendszerre lett elkészítve, viszont más platformokon is működik. A legmegfelelőbb verziók vagy böngészők a különböző platformokon a következők:

- *Android*: 5.0 vagy újabb
- *Web*: Google Chrome, Mozilla Firefox, Microsoft Edge aktuális verziói

Az alkalmazás fejlesztése során fontos szempont volt, hogy különböző képernyőméretek és eszközökön is megfelelő felhasználói élményt nyújtson.

- **Tárhely** Az alkalmazás telepítéséhez és működéséhez legalább 250MB szabad tárhely szükséges. Később ez nőhet, eltárolt hangfájlok vagy gyorsítótározott adatok mennyiségétől függően.

- **Internetkapcsolat**

Aktív internetkapcsolatra van szükség az alkalmazás működéséhez, mivel a mesterséges intelligenciával történő kommunikáció, a beszédfelismerés, valamint az adatok szinkronizálása szerveroldalon történik. Internetkapcsolat hiányában az alkalmazás bizonyos funkciói nem lesznek elérhetőek.

- **Teljesítmény**

Egy nagyon fontos követelmény a gyors válaszidő. Különösen fontos szempont ez a beszélgetések során, ahol a felhasználó valós idejű kommunikációs élményt vár el. Az AI válaszgenerálásának, a beszédfeldolgozásnak és az adatbázislekérdezéseknek optimalizált módon kell működnie, azért hogy az alkalmazás folyamatos és gördülékeny felhasználói élményt biztosítson.

- **Megbízhatóság**

A fejlesztés során fontos szempont volt a stabil működés és a hibák megfelelő kezelése. Különböző hibakezelési mechanizmusokat alkalmazunk annak érdekében, hogy egy esetleges hiba ne okozza az alkalmazás leállítását. A backend oldalo központi hibakezelés került kialakításra, amely lehetőséget nyújt megfelelő válaszok visszaküldését a kliens számára.

- **Skálázhatóság** A rendszer úgy lett kialakítva, hogy lehetővé tehesse a további funkciók és szolgáltatások későbbi integrálását. Az AI szolgáltatások külön microserviceként lettek elkészítve, így a rendszer könnyen bővíthetővé és fejleszthetővé vált. [LF14] Ez lehetőség az új modellek, nyelvek, tanulási funkciók hozzáadására, anélkül hogy a teljes rendszer architektúráján módosítani kellene.

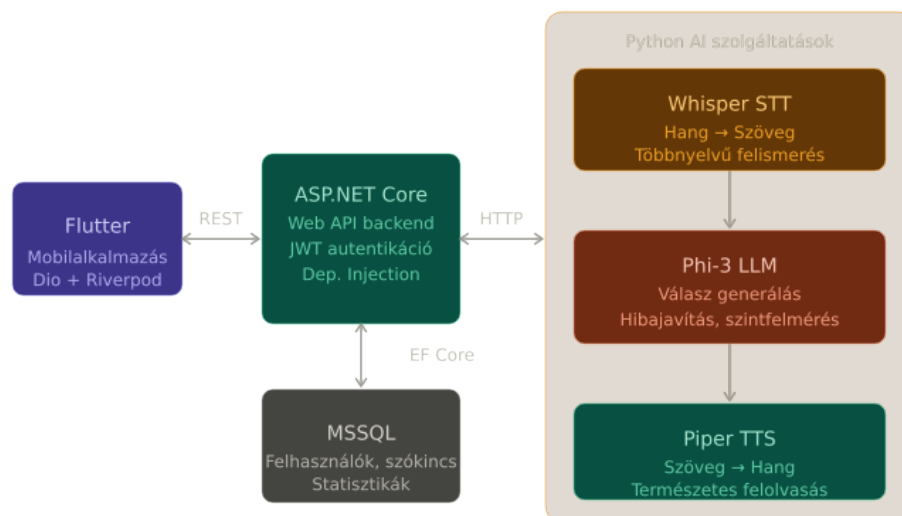
- **Modularitás** Az alkalmazás, köszönhetően a funkciók egymástól elkülönítésének, moduláris felépítést alkalmaz. Ez megkönnyíti a komponensek fejlesztését, tesztelését, cseréjét. A frontend, backend, AI szolgáltatások mind különálló részekként működnek a rendszerben.
- **Bővíthetőség** A rendszer tervezésénél a jövőre is gondoltam. Úgy lett megtervezve, hogy később új gyakorlási módokkal, új AI funkciókkal vagy további nyelvekkel bővíthető legyen. Ez a megközelítés lehetővé teszi az új modulok hozzáadását, anélkül hogy jelentős változásokat kellene végrehajtani a rendszerben.

4. fejezet

Tervezés

4.1. Szoftver terv

A rendszer architektúrája a következő komponensekből tevődik össze: felhasználói interfész, backend API, mesterséges intelligencia szolgáltatásai, adatbázis. Ez a több komponenses felépítés működtetéséhez szükséges az internetkapcsolat, amely a kommunikációhoz segít hozzá a komponensek között. A rendszer architektúráját a következő ábra szemlélteti, ahol megfigyelhető a részek közötti kapcsolat és adatáramlás.



4.1. ábra. Rendszer architektúrája

4.1.1. Logikai nézet

A felhasználói felület egy Flutter framework segítségével fejlesztett mobilalkalmazás. Ez lehetővé teszi a cross platform alkalmazások készítését, így az alkalmazásunk amelyet eredetileg mobilra fejlesztettem, könnyedén futtatható weben, akár desktopon is. A felhasználók ezen a területen érhetik el az alkalmazás funkcióit, mint a beszélgetéseket, gyakorló feladatokat, fejlődésükről való statisztikákat.

A backend rétegért a .NET alapú Web API felelős, amely üzleti logikát, autentikációt, adatkezelést, frontend és adatbázis közötti kommunikációt is végrehajt. Az API fogadja a kliens kéréseit, feldolgozza azokat, majd visszaküldi a megfelelő válaszokat.

A rendszer lelke, legfontosabb része: az API szervíz, amely Python alapú réteg. Itt valósulnak meg a mesterséges intelligenciával kapcsolatos funkciók. Ez a komponens felelős többek között a beszédfelismerésért, a nyelvi hibák felismeréséért és javításáért, válasz generálásért, nem utolsósorban pedig a szöveg felolvasásáért. Az AI szolgáltatások külön komponensekként működnek, amelyekkel a backend API kommunikál. [LF14] Az AI szolgáltatások, amelyek a beszélgetéshez hozzájárulnak azok a Whisper, Phi3, Piper szolgáltatás. A konkrét flow, ami végigmegy ezeken az AI-okon: amikor a felhasználó beszél a mesterséges intelligenciához, akkor a Whisper dolgozza fel a beszédet, amelyet átalakít szöveggé, ezt a szöveget átadja a Phi3 LLM modellnek [RKX⁺22, SYS26, AJA⁺24, Rha26], amely többek között a választ generálja a szövegre, így szimulálva a valós beszélgetést, ezt a választ viszont szöveggé adja vissza a Phi3 modellünk, így végezetül a Piper AI modell alakítja át a szöveget hanggá, biztosítva a valós beszélgetés érzetét.

Az adatok tárolásához egy MSSQL adatbázist használunk. Itt kerülnek eltárolásra a felhasználói adatok, beszélgetés címek, hibák, gyakorlási adatok és statisztikák. Az adatbázis kezeléséhez Code First megközelítést alkalmaztam Entity Framework segítségével, amely lehetővé teszi az adatmodellek egyszerű kezelését és migrációk használatát. [Mic23]

A komponensek közötti kommunikáció internetkapcsolat segítségével megvalósított. A felhasználói felületről érkező kérések először a backend APIhoz kerülnek, amely szükség esetén kommunikál az AI szolgáltatásokkal vagy az adatbázissal. A feldolgozott válasz ezután visszakerül a kliens alkalmazásba.

4.1.2. Folyamat nézet

Az alábbiakban néhány lényeges folyamatot mutatok be, hogy hogyan viselkednek különböző részei a rendszernek.

1. Hangalapú beszélgetés

- A felhasználó hangüzenetet küld az alkalmazás számára
- A hangfájlt elküldjük az API-n keresztül az AI szolgáltatáshoz
- A továbbítás után speech to text AI modell szöveggé alakítja a beszédet
- Az AI feldolgozza a szöveges üzenetet, ehhez választ generál és megállapítja a hibákat, amelyeket json formátumban küld vissza az endpointban
- A generált választ text to speech AI által hanggá alakítjuk, ezt eltároljuk egy mappába
- Az elkészült válasz visszakerül a felhasználói felületre és a hang megszólal, de újrajátszható is lesz a hang egy gomb segítségével, és a szavakat is fel lehet mondítani az AI segítségével egy kattintással.

2. Hibák felismerése és eltárolása

- Amikor a felhasználó az AI modellel beszélget, akkor az AI elemzi a felhasználó válaszai helyességét.

- Ha nyelvi hibát észlel automatikusan eltárolja azt a hibák közé, csoportosítja azt, és magyarázatot is generál
- Ez majd a gyakorláson alapuló tanuláshoz járul hozzá.

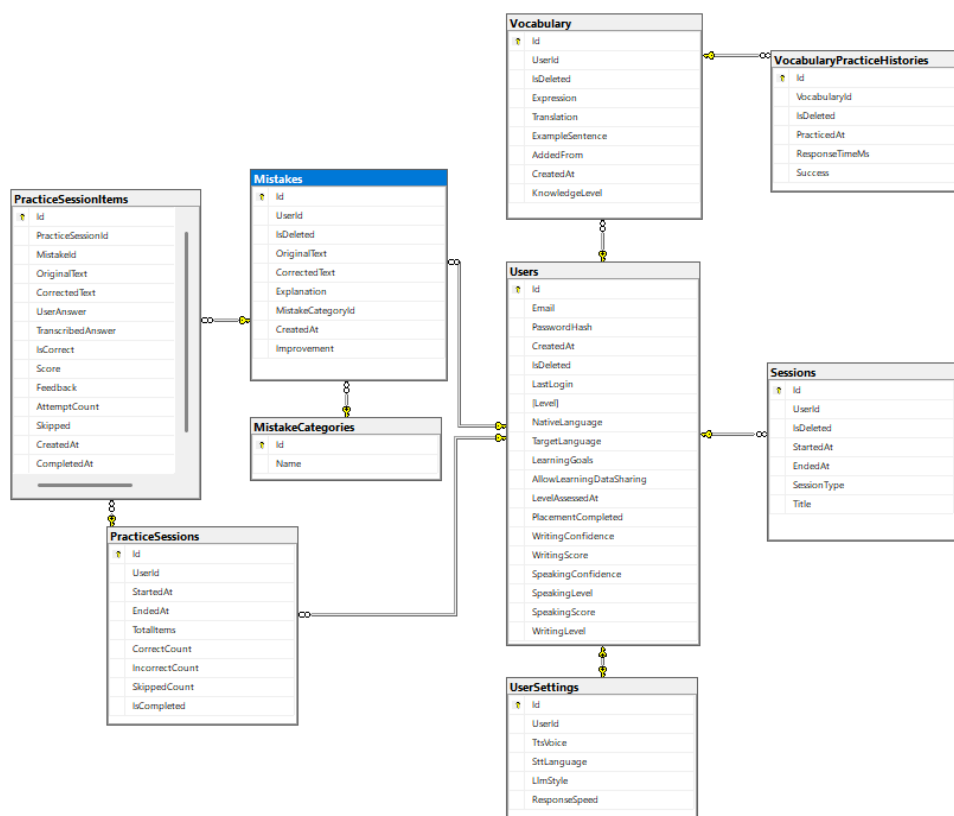
3. Gyakorló feladat generálása

- A rendszer lekéri a felhasználó hibáit az adatbázisból
- A felhasználó tud írásban vagy hangban is gyakorolni, hangban azért nehezebb de hatékonyabb, mert az AI érzékeny az akcentusra és a hanglejtésre is
- Gyakorlatilag a felhasználónak ki kell javítani az előzőekben elkövetett hibáit
- A felhasználó válaszát az AI javítja ki
- Ezen gyakorlatok eredményei eltárolódnak az adatbázisban
- Ezek a gyakorlatok hozzájárulnak a felhasználó fejlődéséhez, az eltárolt adatai pedig a statisztikákhoz és mérésekhez

4. Statisztikák frissítése

- A rendszer a beszélgetések és gyakorlások során különböző adatokat gyűjt a felhasználó aktivitásáról
- Ezeket az adatokat az adatbázisban tároljuk el
- Az API feldolgozza az adatokat és statisztikákat készít, mint például a hibák száma, gyakorlások száma, fejlődési tendencia
- A statisztikák megjelennek a felhasználó profilján és statisztikai felületén

4.2. Adatbázis terv



4.2. ábra. Adatbázis terv

Az adatbázist relációs adatbázisként terveztem meg, amely mindössze 9 táblával rendelkezik. Ezekben a táblákban tárolom el a felhasználó adatait, gyakorlatait. Ezek hozzájárulnak az applikáció megfelelő működéséhez.

Ezekben a táblákban a legnagyobb a felhasználó adatait tartalmazó tábla, ez egy nagyon fontos hely, amiben sok lényeges adat van eltárolva, ezek mind hozzásegítenek a fejlődés méréséhez és magához a fejlődéshez is, mivel az AI ehhez mérten válaszol neki.

A Vocabulary táblában mentődik el minden olyan szó vagy szókapcsolat, amelyet a felhasználó nem ismert és ezeknek fordítása. Ezeket a szavakat elmentheti a felhasználó manuálisan, ha bárhol is szembejön vele és szeretne egy olyan helyre menteni, ahol majd előveheti és gyakorolhatja azt, illetve a mesterséges intelligenciával történő beszélgetés során is hozzáadhat ehhez a táblához. A VocabularyPracticeHistory tábla a Vocabulary-ba eltárolt kifejezések tanulásához járul hozzá. Mivel flashcardok segítségével gyakorolhatja ezeket a felhasználó, itt tárolódik el a helyessége, a válaszidője, és minden más fontos adat, amelyet felhasználhatunk a fejlődés elemzésére.

A Mistakes tábla az azok a szókapcsolatok vagy mondatok tárhelye és azoknak javított verziója, amelyeket a felhasználó elhibázott beszélgetés közben. Mindez úgy tárolódik el ebbe a táblába, hogy beszélgetés közben a mesterséges intelligencia figyeli, milyen hibákat vét a felhasználó és mindezt elmenti kijavított verziójával együtt és magyarázattal. Ezeket a hibákat nem hiába mentjük el, ugyanis gyakorló feladatokkal megtanítjuk a helyes verzióját ezeknek. A PracticeSessions és PracticeSessionItems ezért is kapcsolódik

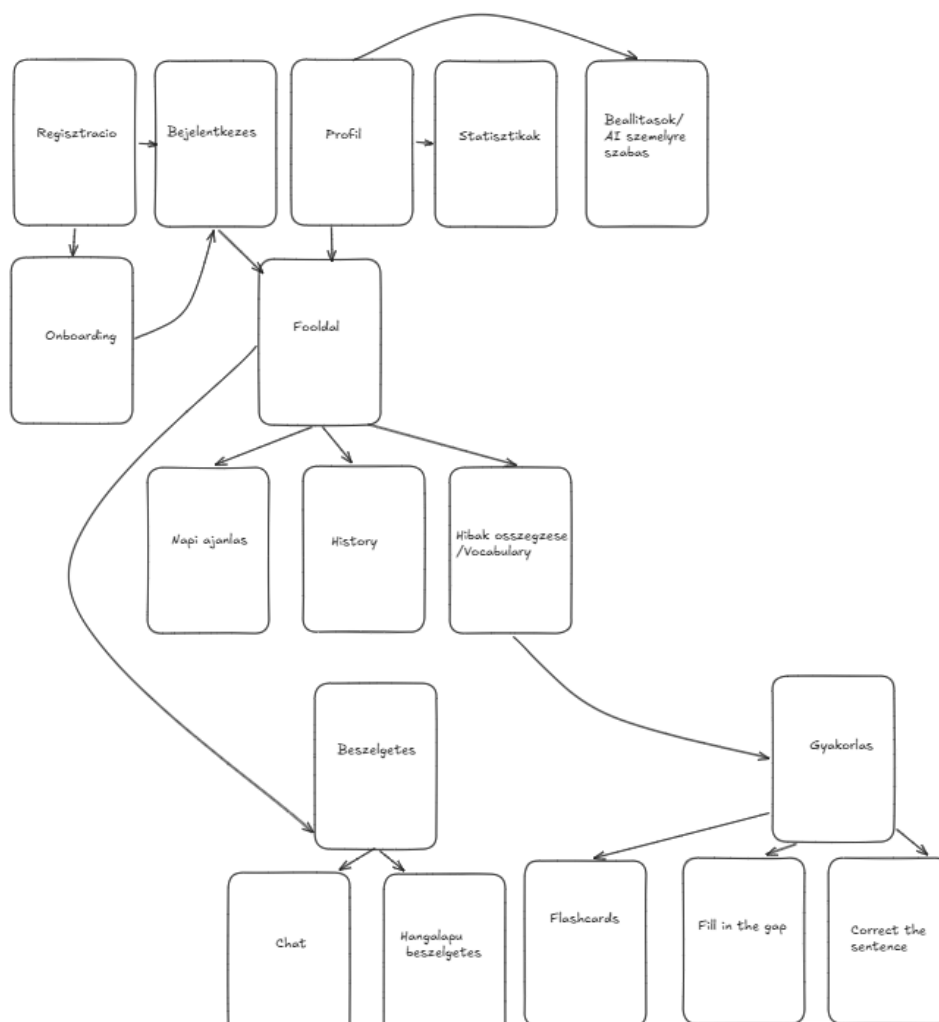
a Mistake táblához, mivel a felhasználó gyakorlását ide tároljuk el, minden egyes adatot róla.

A Sessions tábla a mesterséges intelligenciával való beszélgetésről tárol összefoglaló címet, mivelhogy a felhasználók biztonsága érdekében nem tároljuk a teljes beszélgetést. Ezen kívül még néhány alapvető adat kerül itt tárolásra, mint a beszélgetés kezdete vagy vége.

Az adatbázis MSSQL segítségével íródott. [Mic26d] Ennek kiválasztásakor a teljesítmény, skálázhatóság és könnyen integrálhatóság volt a szempont. A .NET keretrendszerrel és a Visual Studioval nagyon jól integrálható ez, így mindenképp a legjobb választásnak bizonyult.

Az adatbázisterv a 4.2 ábrán látható.

4.3. Felhasználói felület terv



4.3. ábra. Felhasználói felület terv

A felhasználói felület tervezéséhez egy egyszerű digitális eszközt használtam. Mivel első sorban nem a felhasználói felület volt számomra a szempont, így inkább egy egysze-

rűbb tervet készítettem az Excalidraw [Exc26] alkalmazás használatával. Az alkalmazás fejlesztésénél ezeket vettem elő, ezek biztosították az irányt, amerre haladt a projekt. Néhol újabb dolgokat is beillesztettem fejlesztés közben, nem ragaszkodtam a kezdetleges terveimhez mindig.

A felület tervezésénél nem volt pontos irány, hogy merre rakjuk a gombokat, vagy milyen színeket használjunk, inkább egy átfogó képet szerettem volna kapni az alkalmazásról. Arra fektettem a hangsúlyt a tervben, hogy milyen oldalak, képernyők legyenek az applikációban és a többi részével fejlesztés közben foglalkoztam.

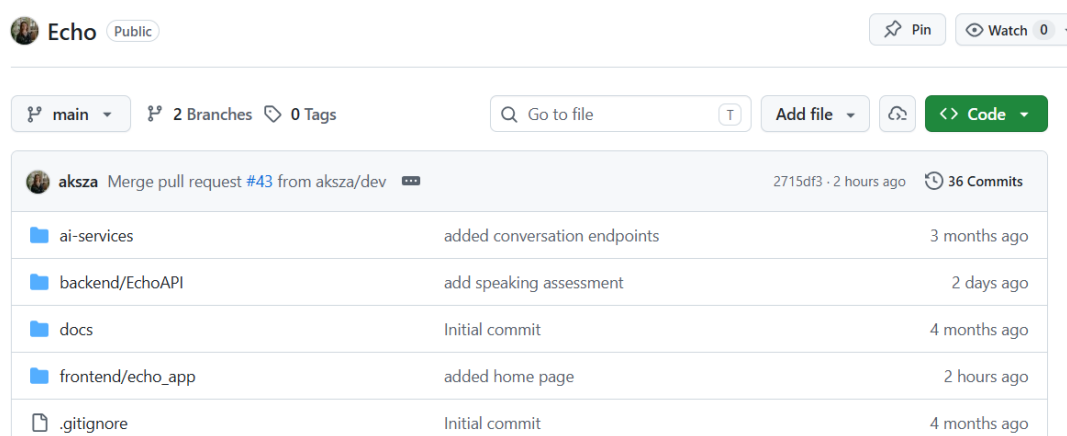
A 4.3 ábrán látható a kezdetleges terv, amelyet még az elején a tervezési fázisban készítettem. Itt látható 17 darab képernyő terv és a nyilak jelölik az azok közötti kapcsolatokat. Mára elmondható, hogy a többsége ezeknek a terveknek megvalósult, és hasonlóan sikerült elkészíteni a kapcsolatokat is, mint ahogyan az ábrán látható. Így elmondható, hogy ez a tervezés mindenképpen megalapozta az elkészítés folyamatát.

Ez a tervezés nem csak a felhasználói felület elkészítésében volt hasznomra, hanem a teljes fejlesztés során. Mivel itt a nyilak segítségével megtekinthető a teljes flow, így nem csak a frontend fejlesztésnél, hanem a backend elkészítésénél is rálátást nyertem arra, hogy melyik feature mi után következzen.

4.4. Verziókövetés

A verziókövetés tekintetében a Github volt számomra egy elengedhetetlen választás. Ide mentettem minden elkészített featuret. Fontosnak éreztem a verziókövető rendszer használatát, ugyanis egy összetett rendszerről beszélünk, amelyben a működő komponenseket jó ha eltároljuk valahol és bármi elakadásnál vissza tudjuk húzni a működő verziót. Ez látható a 4.4 ábrán. [Git26a]

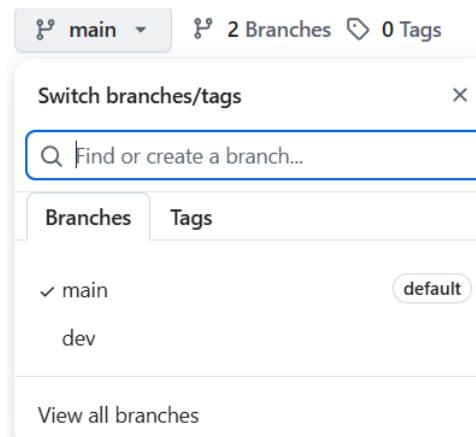
Egy repository segítségével kezeltem a projektet, amelyben három nagy komponens szerepel: a backend, frontend és az ai service.



4.4. ábra. Verziókövető rendszer

A hatékony programozáshoz készítettem egy branchet is, hogy a main branch tisztán maradjon mindig, és a csak a működő feature komponensek legyenek itt, ne a félkészek. A dev branchen fejlesztettem mindent. Itt több commit is volt készítve minden featurehöz. Amikor sikerült elkészíteni egyet, pull request lett nyitva, itt még egyszer

megvizsgáltam a kódot, ha találok benne hibát. Amikor jónak tűnt, máris befésültem a mainre. Ezzel a megközelítéssel könnyedén és tisztán ment a fejlesztés. A branch ábrája a 4.5 megtekinthető.



4.5. ábra. Branch-ek

4.5. Projektmenedzsment

Próbáltam úgy megoldani a projektmenedzsmentet is, hogy azonos platformon legyen a verziókövetőnkkel. Így esett arra a választás, hogy a Github issues menüpontját használjam. [Git26b] Ez arról szól, hogy minden feature számára nyitok egy issue-t. Az issue-k tartalmazhatnak subissue-kat is, ha túl nagy issue jött létre.

Ez számomra azért volt a legkézenfekvőbb, mivel egy platformon elérhetem a kész kódot, a feature terveket és pull request formájában a már kész, elfogadásra váró feature-öket.

A flow az volt, hogy a tervezés résznél megírtam itt, issue formájában minden egyes feature-t, amelyet le kell implementálni. Ekkor meg lehetett itt adni, hogy ki kell elkészítse ezt - ez nagyobb csapat esetében nagyon hasznos lehet, de nálam értelemszerűen én voltam megjelölve itt. Ezután labelt, milestone-t is lehetett hozzáadni, viszont én az egyszerűség kedvéért ezeket a részleteket hanyagoltam. Miután elkészült a tervezés részben az összes issue, kezdődött a fejlesztés. Kiválasztott feature-t leimplementáltam, ezután pedig pull requestet nyitottam. Itt méginkább bebizonyosodott, hogy jó választás volt az, hogy ugyanazon a platformon verziózok és menedzselem a projektet is, ugyanis van egy olyan funkció, hogy a pull request leírásában ha hivatkozunk az adott issue-ra, akkor a pull request merge végrehajtása után automatikusan lezárja az issue-t. Így nem kellett manuálisan lezárnom minden elkészített feature-höz tartozó issue-t.

Az Github oldalán pedig bármikor megtekinthetőek voltak a nyitott vagy már lezárt issue-k. Ezeket bármikor újra lehetett nyitni, vagy akár a nyitottakat lezárni, duplikálni jelölni. Úgy gondolom, hogy ez egy olyan projekthez, amit egy személy készít, bőven elegendő projektmenedzsment eszköznek, mivel minden szükséges kellék megvan benne.

aksza / Echo

Issues 15 Pull requests Agents Discuss

4 we'll start using GitHub Copilot interaction data for AI model tra

is:issue state:open

☐ Open 15 ☐ Closed 15

- ☐  Frontend - AI session ending, scoring
#36 · aksza opened on Jan 9
- ☐  Frontend - AI session screen
#35 · aksza opened on Jan 9
- ☐  Frontend - Ai settings for users
#34 · aksza opened on Jan 9
- ☐  Frontend - Flashcard screen and scores
#33 · aksza opened on Jan 9
- ☐  Frontend - Add to vocabulary
#32 · aksza opened on Jan 9
- ☐  Frontend - Edit Vocabulary
#31 · aksza opened on Jan 9
- ☐  Frontend - Edit profile
#30 · aksza opened on Jan 9

4.6. ábra. Projektmenedzsment

5. fejezet

Megvalósítás

5.1. Az alkalmazás architektúrája

Az alkalmazás fejlesztése során fontos szempont volt egy jól struktúrált, könnyen bővíthető és karbantartható architektúra létrehozása. A rendszert három fő komponensből építettem fel: frontend réteg, backend réteg és AI service réteg. Ezek a komponensek mind egymástól elkülönítve működnek, kommunikációjuk HTTP alapú API hívásokon keresztül történik.

5.2. Frontend megvalósítása

Az alkalmazás kliensoldali része Flutter keretrendszer használatával készült. [Goo26] Ez lehetőséget biztosít modern, platformfüggetlen mobil vagy akár web és desktop alkalmazások fejlesztésére. A Flutter előnye nem más, mint hogy egyetlen kódbázisból több platformra is készíthető alkalmazás. Az Echo alkalmazásban a frontend réteg biztosítja a kapcsolatot a felhasználó és a rendszer többi komponense között, ezen keresztül történik minden fontos funkció elérése, mint a regisztráció, beszélgetés, gyakorlás, hibák és szókincs megtekintése.

A projekt felépítésénél első számú szempont az átláthatóság volt, ezért a fájlokat különböző mappákba csoportosítottuk, funkcionális egységek szerint elrendezve. A *lib* mappában három fontos rész különíthető el: *core*, *features*, *shared* mappa. Ez a szerkezet hozzájárult ahhoz, hogy az alkalmazás részei jól elkülönüljenek, ugyanakkor könnyen bővíthetők legyenek újabb funkciókkal.

A *core* mappa tartalmazza az egész alkalmazás működéséhez szükséges elemeket. Ide tartoznak a konstans értékek, globális providerek, téma beállítások. A hálózati kommunikáció megvalósításához Dio csomagot [Dar26] használtam, amellyel HTTP kéréseket tudunk küldeni a backend felé. Az API végpontok elérhetősége *.env* fájlban van eltárolva flutter dotenv csomag segítségével. [Flu26a]

A *features* mappa tartalmazza az alkalmazás fő funkcióit. Minden feature külön mappába került. Ez a feature alapú felépítés nagyon hasznos, mivel minden funkcionalitás saját egységként kezelhető. A fejlesztés során nagyon megkönnyíti így a dologt, mivel könnyebben átlátható az, hogy melyik működéshez melyik fájl tartozik.

Például, a *conversation* modul külön *data* és *presentation* rétegre oszlik. A *data* réteg az API kommunikációhoz és adatmodellekhez kapcsolódó fájlokat tartalmazza. Ezek

felelősek a backenddel való kommunikációért, mint például az üzenetek kezeléséért, valamint hangalapú beszélgetések válaszainak feldolgozásáért. A *presentation* réteg viszont a felhasználói felülethez kapcsolódó komponenseket tartalmazza.

Állapotkezelés szempontjából a *flutter riverpod* [Riv26] csomagot vettem használatba. Ez a csomag hozzájárul ahhoz, hogy az alkalmazás különböző részei könnyen elérjék és frissítsék a szükséges adatokat, például a bejelentkezett felhasználó adatait, a beszélgetések állapotát vagy a betöltési folyamatokat. Ennek az előnye az, hogy jól illeszkedik a moduláris felépítéshez, és hozzájárul az állapotkezeléshez.

Az alkalmazásban az egyik fontos funkcionalitás a hangalapú kommunikáció. Erre három különböző csomagot használtam fel. A *record* segítségével a felhasználó hangot tud rögzíteni, amelyet később továbbíthatják a backend felé, feldolgozásra. Az *audioplayers* csomag a hangválaszok lejátszására szolgál, míg a *path provider* a fájlok ideiglenes vagy helyi tárolásában nyújt segítséget. [Ilf26, Blu26, Flu26b]

Mégeggy fontos mappa a *shared* mappa. Itt olyan újrafelhasználható elemeket tárolunk, amelyeket több képernyőn is megjeleníthetünk. Ide tartoznak a közös widgetek, amelyekkel egységessé varázsoljuk a felületet. Ennek a mappának segítségével csökkentem a kódismétlést és megkönnyítette a későbbi módosításokat.

Összességében a frontend megvalósítása során egy jól struktúrált, funkcionalitás alapú Flutter alkalmazás jött létre, amely jól kezelhető, képes kommunikálni megfelelően az APIval, valamint támogatni a szöveges és hangalapú nyelvtanulási folyamatokat.

5.3. Backend megvalósítása

Az alkalmazás szerveroldali komponensét ASP.NET Core Web API használatával készítettem. [Mic26c] Itt valósul meg a felhasználói hitelesítés, üzleti logika, adatbázissal való kommunikáció, illetve a mesterséges intelligenciát biztosító szolgáltatásokkal való kapcsolat fenntartása. A fejlesztés során fontos szempont volt a kód átláthatósága, karbantarthatósága, így esett a választás a Clean architektúrára. [Mar17]

A projekt négy fő rétegre bontható: API, Application, Core és Infrastructure. Ezek a rétegek mind elkülönült felelősséggel rendelkeznek, kevés függőséggel, így egyszerű, hosszútávú fejlesztéséhez járul hozzá.

- **API réteg:**

Az API réteg felelős a kliens alkalmazás és a backend közötti kapcsolat létesítésében. Itt HTTP kérések segítségével kommunikálhatnak egymással.

Különböző mappák találhatók ebben a rétegben, mindegyiknek különböző, de nagyon fontos szerepe van a projektben.

A Controller mappában különböző végpontok tárolódnak el. Ez a mappa ezeknek a végpontok kezeléséért, tárolásáért, rendszerezéséért felelős. Többek között ezek a végpontok biztosítják a felhasználói hitelesítést, beszélgetés kezelést, beállításokat, és még sok egyebet.

A DTO mappában szerepelnek azok az osztályok, amelyek az adatátvitelt szolgálják a kliens és szerver között. Ezek az osztályok nagyon fontos szerepet játszanak a projektben, mivel hozzájárulnak ahhoz, hogy csak a szükséges adatokat továbbítsuk a kliens számára. Ezáltal növeljük a rendszer biztonságát és rugalmasságát.

A Mappings mappa tartalmazza az AutoMapper [Aut26] konfigurációit. Itt automatikusan átalakítjuk az entitásokat DTO objektumokká és ugyanúgy fordítva. Ez olyan szempontból hasznos, hogy leredukálja a manuális konvertálások számát, egyszerűsíti a fejlesztést, így időt spórol ugyanakkor van egy központi helye ezeknek az átalakításoknak.

Van egy Middleware [Mic26a] is a rendszerben, amelyben a globális hibakezelést valósítottam meg. Ennek feladata, hogy központilag kezelje az előforduló hibákat és egységes formátumú válaszokat küldjön a kliens számára.

- **Application réteg**

Ez a réteg tartalmazza az alkalmazás üzleti logikáját. Itt olyan szolgáltatásokkal találkozhatunk, amelyek a különböző folyamatok működését valósítják meg.

Itt történik például a felhasználók kezelése, hibák eltárolása, AI szolgáltatásokkal kapcsolatos implementációk. Itt nem szabad adatbázis specifikus megvalósításokat tárolni, kizárólag üzleti logikát, és interfészek segítségével valósul meg a kommunikáció a külső komponensekkel.

- **Core réteg**

Ez az a réteg, amely a rendszer legbelső része. Itt vannak az alkalmazás alapvető elemei, domain logikái.

Az Entities mappa az adatbázis entitásait tárolja, ezek az üzleti objektumok. Ezeknek segítségével és az Entity Framework segítségével jöttek létre code first módon az adatbázis táblái.

Az Enums mappában tároljuk a felsorolásokat, ennek segítségével egységes kezelési módot alakítunk ki az különböző állapotok és kategóriák között.

Az Interfaces mappa egy nagyon fontos hely, mivel ezek határozzák meg a szolgáltatások működését. A dependency injection ennek segítségével valósul meg, amely lehetővé teszi azt implementációk egyszerű cseréjét és a rendszer jobb tesztelhetőségét. [Mic26b]

- **Infrastructure réteg**

Itt a külső szolgáltatások és adatbázis eléréseket valósítom meg.

A Data mappa szolgál az adatbázis kapcsolat konfigurációjában, a DbContext osztály pedig a táblák beállításait és a kapcsolatok kezeléseit biztosítja.

A Repositories mappa olyan osztályokat tartalmaz, amelyek az adatbázisműveletek implementációit tartalmazza. Ezek az osztályok lekérdezéséért, módosításáért, létrehozásáért és törléséért felelősek. A repository minta lett itt alkalmazva, amely elkülöníti az adatkezelést az üzleti logikától.

A Services mappában olyan szolgáltatások vannak, amelyek külső rendszerekkel kommunikálnak. Itt leginkább a repository osztályokat hívjuk, illetve itt valósul meg az AI szolgáltatások hívása is.

Nem utolsó sorban a Utils mappát is megtalálhatjuk itt, amely különböző segédosztályokat tartalmaz, amelyeket bárhol a programban felhasználhatunk.

- **Program konfiguráció**

A rendszer indulásakor a Program.cs fájl központi szerepet játszik. Ez végzi el az alkalmazás konfigurációját. Itt állítódik be az adatbázis kapcsolat, dependency injection konfigurálása, jwt hitelesítés beállítása, middlewarek regisztrációja. [JBS15]

5.4. AI service megvalósítása

A mesterséges intelligenciához kapcsolódó funkciók külön Python alapú mikroszolgáltatásokként lettek megvalósítva. [LF14] Ez a felépítés nagyon előnyös, mivel így elkülönül a .NET backend az AI funkcióktól. Ezzel külön fejleszthetővé válik minden rész, szükség esetén könnyen módosítani lehet minden részt, vagy akár cserélni is.

Az AI réteg három fő szolgáltatásból áll össze: beszédfelismeréshez a Whisper service járul hozzá, a nyelni modell a Phi3 service, a szövegfelolvasást pedig a Piper service végzi.

Mindhárom szerviz hasonló projektstruktúrával rendelkezik. Mindhárom mappában külön mappa van az API útvonalaknak, konfigurációs fájloknak, adatmodelleknek és a logikát tartalmazó osztályoknak. Az api mappa endpointokat tartalmaz, a core konfigurációkat, models a kérés és válaszmódelleket, és a service az adott AI funkció tényleges megvalósítását.

Elsőszámú AI service a Whisper, amely azért felelős, hogy a beszédet szöveggé alakítsa. Itt faster whispert használtam, amely egy gyors változata a modellnek.

[RKX⁺22, SYS26] Az alkalmazásban ez lesz az a komponens, ami elsőként hívódik meg, amikor a felhasználó a mesterséges intelligenciához beszél, ez pedig szöveges formátummá fogja átalakítani a következő lépéshez.

A Phi3 service a nyelvi intelligenciát biztosítja. Ez beszélgetésre és utasításkövetésre optimalizált nyelvi modell. Feladata az, hogy a megadott szövegre utasítás szerinti válaszokat generáljon, felismerje és kijavítsa a nyelvi hibákat a visszaadott szövegben. [AJA⁺24]

A Piper service a harmadik fázishoz járul hozzá a válaszadásban, amikor a szöveges választ, amelyet a Phi3 generált, beszéddé alakítja. Ez azért fontos, mert egy valós beszélgetést szerettem volna leszimulálni, hogy a felhasználó könnyebben memorizálja és hozzászokjon, milyen a valódi kommunikáció. [Rha26]

A backend API HTTP kéréseken keresztül kommunikál ezekkel a mikroszolgáltatásokkal.

6. fejezet

Kutatási terv és értékelési módszertan

A dolgozat gyakorlati részében az kerül vizsgálatra, hogy milyen az elkészített AI alapú szoftver mennyire járul hozzá a tanulók témaspecifikus szókincsfejlődéséhez, beszédképességéhez, hibafelismeréséhez és tanulási motivációjához a hagyományos tanítási módszerhez viszonyítva. Mindezt egy rövidtávú kontrollcsoportos pilot vizsgálat során derítjük ki.[VTH01]

A vizsgálat egy 40 perces angol nyelvű tanóra keretében valósul meg. Ennek témája a technológiahasználat és az online biztonság, amelyben kitérünk a kártékony szoftverek típusaira is. A tananyag sok új, fontos fogalmat dolgoz fel, mint például: virus, worm, tojan horse, randomware, personal information, password, suspicious link. A választott téma alkalmas a szókincsfejlesztés, a szövegértés és beszédalapú gyakorlás vizsgálatára is.

A kutatásban összesen 16 tanuló vesz részt, akik egy magántanár diákjai. A tanulók életkora 12 és 16 év között mozog, nyelvi szintjük pedig A2 és B2 közötti. A résztvevők két 8 fős csoportra osztódnak, amiben vegyesen van gyengébb és erősebb szintű tanuló is. A kontrollcsoport hagyományos tanítási módszerrel dolgozza fel a témát, míg a kísérleti csoport AI alapú alkalmazás segítségével tanul.

A vizsgálatnak a mérésére használunk előtesztet, utótesztet [CS63], beszédképesség mérést és kérdőívet. Az előteszt célja a tanulók kiindulási tudásszintjének mérése, míg az utóteszt a tanulási folyamat után elért eredmények mérésére szolgál. A beszédképesség vizsgálata rövid, irányított beszélgetések alapján történik, míg a kérdőív a tanulói élményt, motivációt és az alkalmazás használhatóságát méri.

6.1. A kutatás célja

A kutatás célja annak vizsgálata, hogy az AI alapú nyelvoktatás milyen szinten támogatja a beszédfejlesztést, szókincs gyarapítást, motivációt, a tanulók rövid távú tanulási eredményei megvizsgálásával, úgy hogy egy konkrét angol nyelvi témát dolgoznak fel.

A kutatás, mivel két különböző módszert vizsgál, összehasonlító jellegű. Az egyik csoport hagyományosabb, tanári irányítással tanul, amelyben prezentáció, vocabulary feladat és beszélgetés az alapja a lecke elsajátításának. A másik csoport ugyanazt a témát boncolgatja, viszont az alkalmazás keretein belül. Itt a leckét elolvashatja, kijelölheti az

ismeretlen kifejezéseket, beszélget az AI aszisztenssel a témáról, amely vizsgálja közben a hibáit, végül pedig a hibákat átismétli.

A kutatás rövidtávú vizsgálatként értelmezhető. Ennek megfelelően az eredmények célja nem általános érvényű következtetéseket von le, hanem annak előzetes vizsgálatára szolgál, azt vizsgálja, hogy az alkalmazás alkalmas lehet oktatási környezetben, és milyen lehetőségeket nyújt a hagyományos módszerekhez képest.

6.2. Kutatási kérdések és hipotézisek

A kutatás során négy darab kérdés tevődik fel, amelyhez kapcsolódik négy hipotézisem.

A kérdések a következők:

1. Milyen mértékben fejlődik a tanulók témaspecifikus angol szóincse az AI alapú alkalmazás használata után?
2. Van-e különbség a hagyományos tanítási módszerrel és az AI alapú alkalmazással tanuló csoport eredményei között?
3. Milyen hatással van az AI alapú alkalmazás használata a tanulók beszédképességére egy adott témáról folytatott beszélgetés során?
4. Hogyan értékelik a tanulók az AI alapú alkalmazás használhatóságát, érthetőségét, motiváló hatását?

A kutatási kérdésekre a következő hipotézisek fogalmazhatók meg:

1. Az AI-alapú alkalmazást használó tanulók témaspecifikus szóincse fejlődik az előteszt és az utóteszt eredményei alapján.
2. Az AI-alapú alkalmazást használó csoport fejlődése nagyobb mértékű lesz, mint a hagyományos módszerrel tanuló kontrollcsoporté.
3. Az AI-alapú alkalmazást használó tanulók beszédképessége javul az adott témáról folytatott irányított beszélgetés során.
4. A tanulók pozitívan értékelik az AI-alapú alkalmazás használhatóságát, érthetőségét és motiváló hatását.

6.3. Résztvevők

A kutatásban 16 résztvevő tanuló van, akiknek életkora 12 és 16 év között van, nyelvi szintjük A2 és B2 szint közötti. [Cou26] A tanulók eltérő előzetes nyelvtudással rendelkeznek.

Két 8 fős csoportra osztottuk őket. A kontrollcsoport hagyományos módszerrel sajátítja el a tananyagot, a kísérleti csoport AI alapú alkalmazást használja. A csoportok úgy vannak kialakítva, hogy kiegyensúlyozott legyen a csoportok nyelvi szintje.

Mivel a kutatásban kiskorú tanulók vesznek részt, az adatok kezelése anonim módon történik. A neveik nem jelennek meg eredményekben, sem adatbázisban, helyette K1-K8 kódokkal jelöljük őket, míg az AI alapú alkalmazást használó csoportot A1-A8 kóddal.

6.4. A tanóra témája és menete

A tanórák 40 percből állnak, amelyeken kétféleképpen fogjuk megtanítani a diákokat a technológiahasználatról, amelyben sok új kifejezést is tanulhatnak és a beszédkészségüket is növelhetik. Ebben a témában fogunk tesztelni.

A kontrollcsoport esetében az óra hagyományosan, tanári magyarázattal és prezentációval történik. Miután ez megvalósult, vocabulary feladatokat kezdenek oldani, utána pedig brainstorming résszel folytatják, amelyben a tanulók a témához kapcsolódó kérdésekre válaszolnak.

A kísérleti csoport ugyanazt a témát dolgozza fel, az applikáció segítségével. Az applikációban megtalálja ugyanazt a tananyagot szöveg formájában, amelyet meg is hallgathat. Itt kijelöli az ismeretlen kifejezéseket, amelyeket később tanulhat. Ezek után AI által vezetett beszélgetés történik, ahol a témáról beszélgetnek. Itt hibavizsgálat folyik a háttérben beszélgetés közben, és ezután a megállapított hibákat újra begyakorolhatja, és elsajátíthatja helyesen a kifejezéseket.

A két tanulási folyamat eltérő, de ugyanarra a tanagyagra készíti fel a tanulókat. Ez lehetővé teszi, hogy egy adott téma tanulási módszerét vizsgáljuk.

6.5. MÉRŐESZKÖZÖK

A kutatás során több mérőeszközt is használok, annak érdekében, hogy a tanulási eredményt több szempontból is megvizsgáljuk. Az értékelés kiterjed szókincsfejlődésre, beszédkészségre, tanulói motivációra is. Ennek megfelelően különböző méréseket veszek igénybe.

Elsősorban írásbeli tesztekkel mérek. Ez előteszt és utóteszt formájában valósul meg, [CS63] ahol a tanulók kiindulási tudásszintjének felmérése történik meg a tanóra előtt és a tanóra utáni megszerzett tudást is felméri. A feladatok között szerepel a szó-jelentés párosítás, mondatkiegészítés, rövid válaszok írása kérdésekre.

A tanulói motiváció és élmény mérése kérdőív kitöltésével történik. A kérdőívben 1 és 5 között kell pontozni az állításokat, amelyek vizsgálják, hogy a tanulók mennyire találják hasznosnak, érdekesnek, motiválónak a tanulási módszert.

Az alkalmazással tanuló csoportoknál adatforrásként szolgál az alkalmazásban rögzített tanulási adatok, mint az ismeretlen szavak száma, vocabulary, helytelen mondatok. Ezek az adatok azt is lehetővé teszik, hogy vizsgáljuk, milyen mértékben használták az alkalmazás funkcióit.

6.6. Adatelemzési módszerek

Az adatok elemzése kvantitatív és kvalitatív szempontokkal történik. A kvantitatív elemzés méri, hogy a tanulók teljesítménye mennyire változik az előteszt és utóteszt között, és azt is hogy a kontrollcsoport és a kísérleti csoport eredményei között megfigyelhető lehet különbség vagy sem.

Elsőszámú mérés a leíró statisztikai mutatók kiszámítása. Itt kiszámoljuk az átlagos, szórást, minimum, maximum értéket az előteszt és utóteszt közötti fejlődésben. Ugyanez alkalmazható a beszédkészség felmérésében is, ahol az előtesztre és utótesztre is kap a tanártól pontot.

A csoportokon belüli fejlődés vizsgálatára párosított t-próba alkalmazható, [Fie13] ahol a tanulók előtesztjei és utótesztjeinek eredményét hasonlítjuk. Ezzel kimutatható, hogy az adott csoportban történt-e statisztikailag kimutatható fejlődés. A kontroll és a kísérleti csoportokat is összehasonlítom, független t-próbával, amely azt vizsgálja, hogy a két módszer között van-e jelentős fejlődési különbség.

A statisztikai szignifikancia mellett a hatásméretet is megmérem a Cohen-féle d értékkel. [Coh88] Itt a két csoport közötti különbséget gyakorlati szempontból vizsgálom. Ez azért szükséges, mert kis minta esetén előfordul, hogy egy különbség nem tűnik statisztikailag szignifikánsnak, viszont pedagógiai vagy gyakorlati szempontl mégis figyelemreméltó tendenciát mutathat ki.

A kérdőíves válaszok elemzése leíró statisztikai módszerekkel történik. A kijelentéseket 1 és 5 között kell pontozzák, ezek alapján pedig az átlagértékből kiszámítható a motiváció, érdeklődés, használhatóság és elégedettség az alkalmazással szemben.

6.7. Etikai szempontok

A kutatás során kiemelt figyelmet szenteltem etikai szempontokra, mivel a vizsgálat során kiskorú résztvevőket mérünk fel. A részvétel önkéntes alapon történik, szülők is beleegyeztek és tájékoztatást kaptak a kutatás céljáról, adatok felhasználásáról és kezeléséről, illetve a tanár is beleegyezett, hogy az órai keretében vizsgálják.

Az adatkezelés végig anonim módon történik. A tanulók nevei sem teszten, sem eredményekben nem szerepelnek. A résztvevők azonosítása kódokkal történik. Az így szerzett adatok kizárólag dolgozat kutatási céljára használható fel és nem alkalmas a tanulók személyes azonosítására.

A beszédkésztség felmérésénél nem készülhet sem hangfelvétel, sem videófelvétel. A beszélgetés során a tanár az előre elkészített értékelési rubrika alapján közvetlenül a beszélgetés során rögzíti a pontszámokat. Ez a megoldás adatvédelmi szempontból lett alkalmazva.

A kutatás etikai szempontból fontos eleme az is, hogy az alkalmazás használata során keletkező adatok csak korlátozott célból kerüljenek felhasználásra. Az adatokat csak tanulási folyamat támogatását és kutatási elemzést szolgálják, harmadik fél számára semmi módon sem kerülhetnek továbbításra.

7. fejezet

Kutatási eredmények és kiértékelés

8. fejezet

Következtetések és továbbfejlesztési lehetőségek

Ábrák jegyzéke

4.1. Rendszer architektúrája	21
4.2. Adatbázis terv	24
4.3. Felhasználói felület terv	25
4.4. Verziókövető rendszer	26
4.5. Branch-ek	27
4.6. Projektmenedzsment	28

Irodalomjegyzék

- [AJA⁺24] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. arXiv preprint arXiv:2404.14219, 2024.
- [Aut26] AutoMapper. Automapper documentation. <https://docs.automapper.io/>, 2026. Látogatás dátuma: 2026-06-13.
- [Blu26] Blue Fire. audioplayers package. <https://pub.dev/packages/audioplayers>, 2026. Látogatás dátuma: 2026-06-13.
- [Coh88] Jacob Cohen. Statistical Power Analysis for the Behavioral Sciences. Lawrence Erlbaum Associates, Hillsdale, NJ, 2 edition, 1988.
- [Cou26] Council of Europe. Common european framework of reference for languages: Level descriptions. <https://www.coe.int/en/web/common-european-framework-reference-languages/level-descriptions>, 2026. Látogatás dátuma: 2026-06-13.
- [CS63] Donald T. Campbell and Julian C. Stanley. Experimental and Quasi-Experimental Designs for Research. Rand McNally, Chicago, 1963.
- [Dar26] Dart Tools. Dio package. <https://pub.dev/packages/dio>, 2026. Látogatás dátuma: 2026-06-13.
- [DSN⁺11] Sebastian Deterding, Miguel Sicart, Lennart Nacke, Kenton O’Hara, and Dan Dixon. Gamification: Using game design elements in non-gaming contexts. In Proceedings of the 2011 Annual Conference Extended Abstracts on Human Factors in Computing Systems, pages 2425–2428. ACM, 2011.
- [Duo23] Duolingo. Duolingo max uses openai’s gpt-4 for new learning features. <https://blog.duolingo.com/duolingo-max/>, 2023. Látogatás dátuma: 2026-06-13.
- [Exc26] Excalidraw. Excalidraw whiteboard. <https://excalidraw.com/>, 2026. Látogatás dátuma: 2026-06-13.
- [Fie13] Andy Field. Discovering Statistics Using IBM SPSS Statistics. SAGE Publications, London, 4 edition, 2013.
- [Flu26a] Flutter Community. flutter_dotenv package. https://pub.dev/packages/flutter_dotenv, 2026. Látogatás dátuma: 2026-06-13.

- [Flu26b] Flutter Team. path_provider package. https://pub.dev/packages/path_provider, 2026. Látogatás dátuma: 2026-06-13.
- [Git26a] Git. About version control. <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>, 2026. Látogatás dátuma: 2026-06-13.
- [Git26b] GitHub. About issues. <https://docs.github.com/articles/about-issues>, 2026. Látogatás dátuma: 2026-06-13.
- [Goo26] Google LLC. Flutter documentation. <https://docs.flutter.dev/>, 2026. Látogatás dátuma: 2026-06-13.
- [JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. Json web token (jwt). <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. RFC 7519. Látogatás dátuma: 2026-06-13.
- [LF14] James Lewis and Martin Fowler. Microservices: A definition of this new architectural term. <https://martinfowler.com/articles/microservices.html>, 2014. Látogatás dátuma: 2026-06-13.
- [llf26] llfbandit. record package. <https://pub.dev/packages/record>, 2026. Látogatás dátuma: 2026-06-13.
- [Lon96] Michael H. Long. The role of the linguistic environment in second language acquisition. In William C. Ritchie and Tej K. Bhatia, editors, Handbook of Second Language Acquisition, pages 413–468. Academic Press, New York, 1996.
- [LSS13] Roy Lyster, Kazuya Saito, and Masatoshi Sato. Oral corrective feedback in second language classrooms. Language Teaching, 46(1):1–40, 2013.
- [Mar17] Robert C. Martin. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Prentice Hall, Boston, 2017.
- [Mic23] Microsoft Corporation. Migrations overview – entity framework core. <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/>, 2023. Látogatás dátuma: 2026-06-13.
- [Mic26a] Microsoft Corporation. Asp.net core middleware. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/>, 2026. Látogatás dátuma: 2026-06-13.
- [Mic26b] Microsoft Corporation. Dependency injection in .net. <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>, 2026. Látogatás dátuma: 2026-06-13.
- [Mic26c] Microsoft Corporation. Overview of asp.net core. <https://learn.microsoft.com/en-us/aspnet/core/overview>, 2026. Látogatás dátuma: 2026-06-13.

- [Mic26d] Microsoft Corporation. Sql server technical documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>, 2026. Látogatás dátuma: 2026-06-13.
- [OWA26] OWASP Foundation. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026. Látogatás dátuma: 2026-06-13.
- [Pra26a] Praktika. Praktika – ai language tutor. <https://play.google.com/store/apps/details?id=ai.praktika.android>, 2026. Látogatás dátuma: 2026-06-13.
- [Pra26b] Praktika. Praktika - learn and speak new languages with ai tutors. <https://praktika.ai/>, 2026. Látogatás dátuma: 2026-06-13.
- [Pyt26] Python Software Foundation. The python tutorial. <https://docs.python.org/3/tutorial/>, 2026. Látogatás dátuma: 2026-06-13.
- [Rha26] Rhasspy. Piper: A fast, local neural text to speech system. <https://github.com/rhasspy/piper>, 2026. Látogatás dátuma: 2026-06-13.
- [Riv26] Riverpod. Riverpod documentation. <https://riverpod.dev/>, 2026. Látogatás dátuma: 2026-06-13.
- [RKX⁺22] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. arXiv preprint arXiv:2212.04356, 2022.
- [SL12] Masatoshi Sato and Roy Lyster. Peer interaction and corrective feedback for accuracy and fluency development. Studies in Second Language Acquisition, 34(4):591–626, 2012.
- [Spe26a] Speak. Speak - the language learning app that gets you speaking. <https://www.speak.com/>, 2026. Látogatás dátuma: 2026-06-13.
- [Spe26b] Speak. Speak: Language learning. <https://apps.apple.com/us/app/speak-language-learning/id1286609883>, 2026. Látogatás dátuma: 2026-06-13.
- [Swa85] Merrill Swain. Communicative competence: Some roles of comprehensible input and comprehensible output in its development. In Susan M. Gass and Carolyn G. Madden, editors, Input in Second Language Acquisition, pages 235–253. Newbury House, Rowley, MA, 1985.
- [SYS26] SYSTRAN. Faster-whisper: Faster whisper transcription with ctranslate2. <https://github.com/SYSTRAN/faster-whisper>, 2026. Látogatás dátuma: 2026-06-13.
- [VTH01] Edwin R. Van Teijlingen and Vanora Hundley. The importance of pilot studies. Social Research Update, 2001. Látogatás dátuma: 2026-06-13.

- [XZS⁺24] Feiwen Xiao, Priscilla Zhao, Hanyue Sha, Dandan Yang, and Mark Warschauer. Conversational agents in language learning. Journal of China Computer-Assisted Language Learning, 4(2):300–325, 2024.